

Lezione 2: Basi di Programmazione Object-Oriented (OOP)

Ingegneria del Software - Sede di Cremona

a.a. 2026/2027

La lezione introduce i concetti fondamentali della programmazione ad oggetti (OOP): classi e oggetti, attributi, metodi, costruttori, creazione e utilizzo di oggetti, associazioni tra classi.

Unified Modeling Language (UML) è una notazione grafica semi-formale sviluppata per uniformare la rappresentazione dei sistemi software da diverse prospettive. Durante la lezione, tutti i concetti verranno introdotti parallelamente tramite UML e il corrispondente codice Java.

1 Classi e Oggetti

La *classe* è il concetto fondamentale del paradigma di programmazione ad oggetti.

► Una **classe** rappresenta lo “stampo” delle diverse entità che esistono all’interno di uno specifico sistema software.

In riferimento al caso di studio PoliFlix, nella lezione precedente abbiamo stabilito che l’applicativo gestirà dati relativi agli utenti, ognuno dei quali ha delle caratteristiche (ad esempio, username e password) e può svolgere delle azioni (ad esempio, registrarsi ed effettuare il login). *Utente* sarà, quindi, la prima classe che andrà a popolare il progetto di PoliFlix.

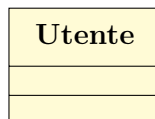


Figura 1: Rappresentazione UML della classe Utente.

```
1 public class Utente {  
2 }
```

Listing 1: Dichiarazione in Java della classe Utente.

► Mentre una classe costituisce lo “stampo”, cioè definisce la struttura e il comportamento dell’entità modellata, le specifiche realizzazioni della classe sono chiamate **oggetti** o **istanze**.

Facendo, ad esempio, riferimento alla classe Utente, gli utenti “Andrea” e “Barbara” effettivamente registrati nell’applicativo costituirebbero due oggetti o istanze distinte della classe Utente.

1.1 Attributi

► Una classe definisce anche quali caratteristiche, propriamente definiti **attributi**, possiede l’entità modellata. Ogni attributo ha un nome (anche detto identificativo) e un tipo.

Ad esempio, un utente è caratterizzato da uno username e da una password che costituiranno, quindi, i primi due attributi della classe Utente. Possiamo assumere che entrambi siano di tipo Stringa.

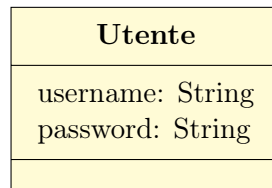


Figura 2: Rappresentazione UML della classe Utente compresi gli attributi.

```
1 public class Utente {
2     String username;
3     String password;
4 }
```

Listing 2: Dichiarazione in Java della classe Utente compresi gli attributi.

1.2 Metodi

► Una classe contiene anche la specifica delle funzionalità che gli oggetti di tale classe possono svolgere, chiamati **metodi**.

La dichiarazione di un metodo è sintatticamente analoga a quella di una normale funzione, ad esempio:

`String restituisciCredenziali(boolean con_pw)`

definisce un metodo chiamato `restituisciCredenziali` che prende in input un booleano `con_pw` e restituisce una Stringa.

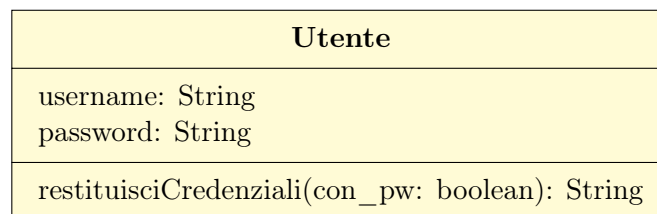


Figura 3: Rappresentazione UML della classe Utente compresi attributi e metodi.

```
1 public class Utente {
2     String username;
3     String password;
4
5     String restituisciCredenziali(boolean con_pw){
6         if (!con_pw) return username;
7         return username + " " + password;
8     }
9 }
```

Listing 3: Dichiarazione in Java della classe Utente compresi attributi e metodi.

Come da Listing 3, all'interno di un metodo è possibile accedere agli attributi tramite il loro identificativo.

1.3 Costruttori

► E' possibile dichiarare oggetti (o *istanziare*) una classe tramite dei metodi appositi denominati **costruttori**.

Il costruttore è l'unico metodo di una classe che ha come identificativo il nome della classe stessa e non necessita di un tipo di ritorno, ad esempio:

`Utente(String u, String p)`

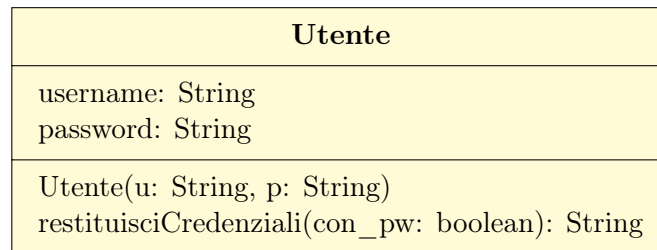


Figura 4: Rappresentazione UML della classe Utente con aggiunta del costruttore.

```
1 public class Utente {  
2     String username;  
3     String password;  
4  
5     Utente(String u, String p) {  
6         username = u;  
7         password = p;  
8     }  
9  
10    String restituisciCredenziali(boolean con_pw){  
11        if (!con_pw) return username;  
12        return username + " " + password;  
13    }  
14 }
```

Listing 4: Dichiarazione in Java della classe Utente con aggiunta del costruttore.

Lo scopo del costruttore è di creare (e, implicitamente, restituire) un nuovo oggetto della classe specificata e assegnare i valori iniziali degli attributi. Ad esempio, l'invocazione `Utente("Andrea", "1234")` creerà un nuovo oggetto della classe Utente con username "Andrea" e password "1234".

1.4 Creazione e utilizzo degli oggetti

Per creare un nuovo oggetto, è necessario invocare un costruttore all'interno del proprio codice preceduto dalla keyword `new`, ad esempio:

`Utente u_1 = new Utente("Andrea", "1234")`

creerà una nuova variabile di tipo Utente (quindi, un oggetto della classe Utente) con username "Andrea" e password "1234".

Una volta creato un oggetto `u`, è possibile accedere ai suoi attributi nel seguente modo:

`u.username`

e invocarne i metodi con:

`u.restituisiCredenziali(true)`

```
1 public static void main(String[] args) {  
2     Utente u_1 = new Utente("Andrea", "1234");  
3     Utente u_2 = new Utente("Barbara", "5678");  
4     List<Utente> utenti = new ArrayList<>();  
5     utenti.add(u_1);  
6     utenti.add(u_2);  
7  
8     for (Utente u : utenti)  
9         System.out.println(u.restituisiCredenziali(true));  
10 }
```

Listing 5: Esempio di creazione e utilizzo degli oggetti.

Notate che, all'interno di un metodo, è possibile accedere a parametri in input, variabili dichiarate localmente, e attributi dell'oggetto su cui il metodo è stato invocato. Ad esempio, con l'istruzione `u_1.restituisceCredenziali(true)` è possibile accedere agli attributi dell'oggetto `u_1` ma non di `u_2`.

In Java, nel corpo di un metodo, è possibile accedere agli attributi dell'oggetto su cui il metodo è stato invocato anche con la forma `this`. (vedi Listing 6).

```
1 public class Utente {
2     String username;
3     String password;
4
5     Utente(String u, String p) {
6         this.username = u;
7         this.password = p;
8     }
9
10    String restituisciCredenziali(boolean con_pw) {
11        if (!con_pw) return this.username;
12        return this.username + " " + this.password;
13    }
14 }
```

Listing 6: Classe Utente con l'utilizzo della keyword `this`.

1.5 Confronto tra Oggetti

L'operatore di confronto `==` su due oggetti `a` e `b` restituisce `true` se `a` e `b` sono lo *stesso* oggetto, non se hanno gli attributi con lo stesso valore. Ad esempio:

```
1 Utente u_1 = new Utente("user", "1234");
2 Utente u_2 = new Utente("user", "1234");
3 Utente u_3 = u_1;
4
5 if (u_1 == u_2)
6     System.out.println("u_1 == u_2");
7 else
8     System.out.println("u_1 != u_2");
9 if (u_1 == u_3)
10    System.out.println("u_1 == u_3");
11 else
12    System.out.println("u_1 != u_3");
```

Listing 7: Esempio di confronto tra oggetti.

stampa `u_1 != u_2` e `u_1 == u_3` poichè `u_1` e `u_3` puntano allo stesso oggetto, mentre `u_1` e `u_2` puntano a due oggetti distinti, seppur con tutti gli attributi con lo stesso valore.

Per definire una relazione di uguaglianza personalizzata, è necessario dichiarare all'interno della classe interessata un metodo `boolean equals(Object o)` che confronta l'oggetto `this` con l'o passato in input. Ad esempio:

```
1 boolean equals(Utente other) {
2     return this.getUsername().equals(other.getUsername()) && this.getPassword().equals(
3         other.getPassword());
4 }
```

In riferimento al Listing 7, `u_1.equals(u_2)`, a differenza di `u_1==u_2`, restituisce `true`.

1.6 Visibilità di attributi e metodi

Nella maggior parte dei casi, è preferibile restringere l'accesso agli attributi di una classe dall'esterno. In altre parole, tipicamente è necessario limitare a quali altre entità del progetto è concesso l'utilizzo della forma diretta `o.attr` per accedere direttamente (in lettura e scrittura) all'attributo `attr` dell'oggetto `o`. Lo stesso discorso si applica ai metodi, per definire in quali classi ne è permessa l'invocazione.

Ciò è possibile modificando la *visibilità* degli attributi anteposto alle loro dichiarazioni le keyword `public`, `private`, o `protected`. Tabella 1 riassume le differenze tra i gradi di visibilità.

Visibilità (Java)	Simbolo UML	Significato
<code>public</code>	+	Elemento accessibile da qualsiasi classe.
<code>private</code>	-	Elemento accessibile solo all'interno della classe stessa.
<code>protected</code>	#	Elemento accessibile dalla classe stessa e dalle sottoclassi.
nessuna keyword	~	Elemento accessibile dalle classi nello stesso package, ma non dall'esterno.

Tabella 1: Gradi di visibilità in Java e loro rappresentazione in UML

E' prassi avere tutti i metodi di una classe pubblici (a meno di eccezioni che ne richiedano un accesso più ristretto) e gli attributi privati. L'accesso in lettura e scrittura agli attributi è permesso solo attraverso opportuni metodi chiamati *getter* e *setter* che implementano tutti i controlli del caso su quali valori siano ammessi e in quali condizioni.

```

1 private String username;
2
3 // getter dell'attributo username
4 public String getUsername() {
5     return this.username;
6 }
7
8 // setter dell'attributo username
9 public void setUsername(String u) {
10     this.username = u;
11 }

```

Listing 8: Esempio di getter e setter nella classe Utente.

D'ora in poi, per preservare la leggibilità dei diagrammi riportati, si assuma che, per ogni attributo privato, la classe contenga anche i rispettivi metodi getter e setter.

2 Associazioni

Vogliamo rendere l'attuale implementazione di PoliFlix più *modulare*, cioè raccogliere tutte le funzionalità affini (ad esempio, riguardanti la gestione degli utenti) nello stesso componente.

A questo scopo, aggiungiamo la classe *ManagerUtenti* che si occupa della manutenzione degli utenti registrati. Questo implica l'esistenza di un'*associazione* tra le classi Utente e ManagerUtenti.

► Definiamo un'**associazione** da una classe A a una classe B se la classe B compare nella dichiarazione della classe A, che sia sotto forma di attributo di A, parametro in input o output di un metodo di A o come variabile locale all'interno di un metodo della classe A.



Figura 5: Associazione tra le classi Utente e ManagerUtenti

L'associazione è *diretta* da ManagerUtenti a Utente perchè ManagerUtenti ha visibilità della classe Utente, ma non è vero il viceversa (la classe Utente non è al corrente dell'esistenza della classe ManagerUtenti).

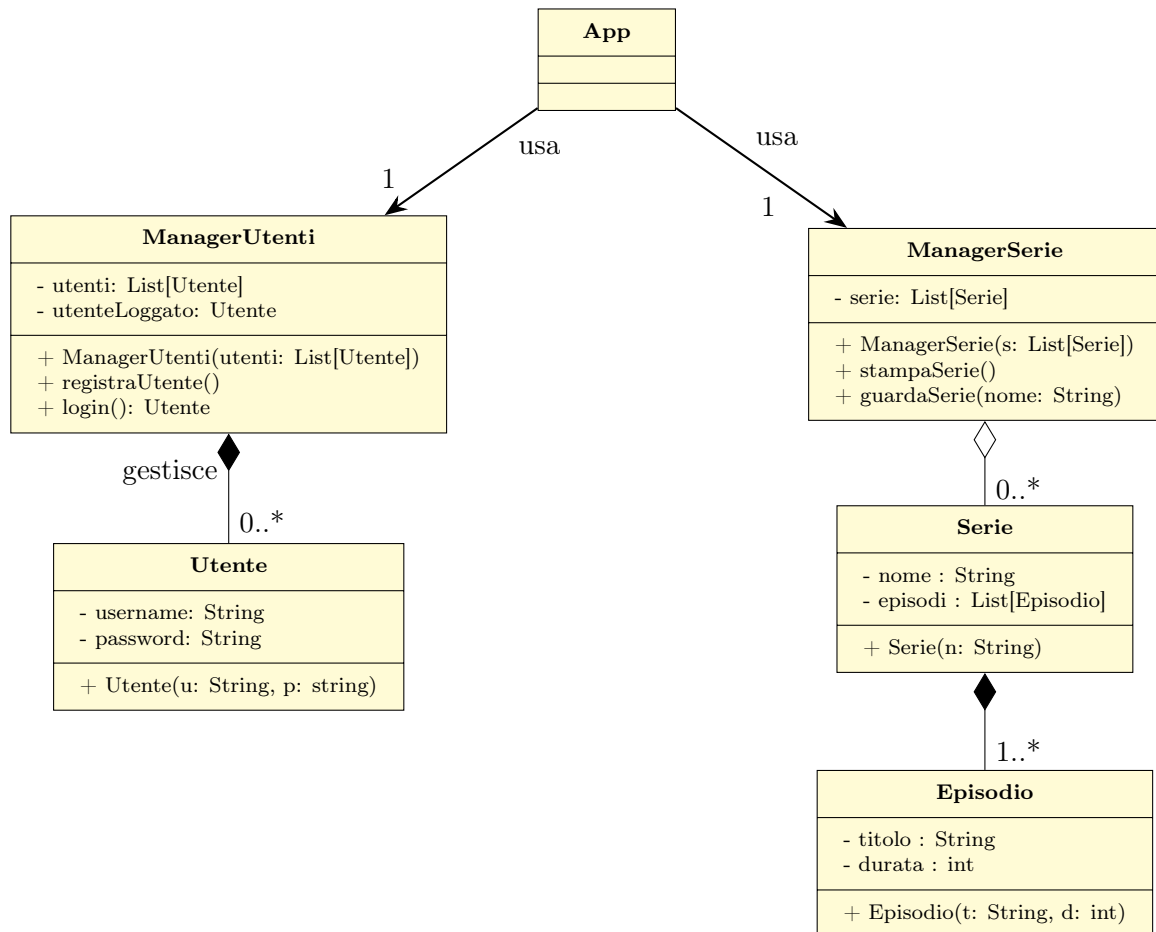


Figura 6: Design di PoliFlix incluse le classi Episodio, Serie, e ManagerSerie.

L'etichetta $0..*$ rappresenta la *molteplicità* dell'associazione, in particolare vuol dire che **ManagerUtenti** può avere visibilità su 0 (se la lista `utenti` è vuota) fino a illimitate istanze della classe **Utente**. Le possibili molteplicità sono riassunte in Tabella 2.

Molteplicità	Significato
1	Esattamente un elemento
0..1	Zero o un elemento
0..*	Nessun limite superiore: da zero a molti elementi
1..*	Almeno un elemento, fino a molti
m..n	Almeno m elementi, al massimo n elementi (es. 2..5)

Tabella 2: Sintesi delle molteplicità UML

2.1 Aggregazioni e Composizioni

Vogliamo estendere il progetto di PoliFlix con il modello delle serie che possono essere riprodotte. Ogni serie è caratterizzata da un nome e da una lista di episodi, dove ogni episodio, a sua volta, è caratterizzato da un titolo e una durata. Tra le classi **Serie** ed **Episodio** esisterà, quindi, un'associazione. In particolare, se una serie viene cancellata, non ha senso che i suoi episodi continuino ad esistere all'interno dell'applicativo: l'associazione tra le due classi costituisce, quindi, una *composizione*.

► Si ha una **composizione** tra una classe A (il contenitore) e una classe B (il contenuto), quando creazione e distruzione del contenuto avvengono sempre all'interno o in concomitanza con quelle del contenitore.

Come per gli utenti, vogliamo raggruppare tutte le funzionalità relative alle serie all'interno di un componente `ManagerSerie`. Il `ManagerSerie` gestisce effettivamente una lista di serie ma non è responsabile della sua creazione o distruzione: ciò costituisce una *aggregazione*.

► Si ha una **aggregazione** tra una classe A (il contenitore) e una classe B (il contenuto), quando creazione e distruzione del contenuto **non** avvengono all'interno o in concomitanza con quelle del contenitore.

La figura 6 mostra il diagramma UML delle classi (o *class diagram*) che costituiscono il progetto attuale di PoliFlix inclusa la gestione delle serie e degli episodi.

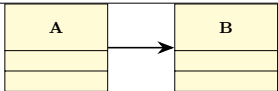
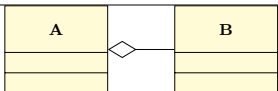
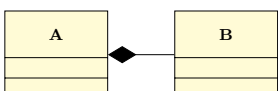
Relazione	Simbolo UML	Significato
Associazione		A utilizza o conosce B. Non implica vincoli di vita o proprietà tra gli oggetti.
Aggregazione		A è contenitore di B, ma B può esistere indipendentemente da A.
Composizione		A è contenitore di B, e B non può esistere senza A. Se un'istanza di A viene eliminata, devono essere eliminate anche le istanze di B che essa contiene.

Tabella 3: Differenze tra associazione, aggregazione e composizione in UML

3 Metodi e Attributi Statici

In Java, un attributo o un metodo può essere dichiarato **static**. Questo significa che l'elemento appartiene alla classe e non a una singola istanza (oggetto). Tutte le istanze della classe, quindi, condividono lo stesso valore dell'attributo statico.

Un attributo statico viene dichiarato con la keyword **static**:

```

1 public class Contatore {
2     public static int numeroOggetti = 0;
3
4     public Contatore() {
5         numeroOggetti++; // incremento condiviso tra tutte le istanze
6     }
7 }
```

Listing 9: Esempio di attributo statico.

Ogni volta che viene creata una nuova istanza di `Contatore`, l'attributo `numeroOggetti` aumenta per tutte le istanze già esistenti. Tutte le istanze leggono e modificano lo stesso valore.

Un metodo statico appartiene alla classe e può essere invocato senza creare un oggetto:

```

1 public class Matematica {
2     public static int quadrato(int n) {
3         return n * n;
4     }
5 }
6
7 public class Main {
8     public static void main(String[] args) {
9         int x = Matematica.quadrato(5); // chiamata senza istanziare la classe Matematica
10        System.out.println(x); // 25
11    }
12 }
```

Listing 10: Esempio di metodo statico.

Ad esempio, il metodo che legge i dati relativi alle serie e i loro episodi a partire da un file e ne crea gli oggetti di conseguenza è un metodo statico della classe Serie (non ha bisogno di un'istanza della classe Serie per essere invocato). In UML, attributi e metodi statici appaiono sottolineati.

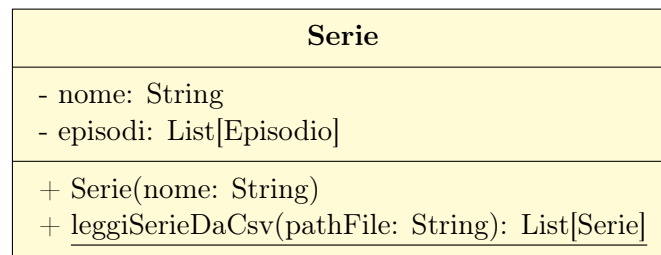


Figura 7: Rappresentazione UML della classe Serie con aggiunta del metodo statico.

► Note importanti:

- Un metodo statico può accedere solo ad attributi e altri metodi statici della classe.
- Non può utilizzare direttamente `this`, perché non esiste un oggetto associato.
- Sono spesso usati per utility, costanti condivise o contatori globali.

4 Gestione File

In Java, la gestione dei file si basa sulle classi del package `java.io`. Le operazioni principali sono lettura e scrittura.

Per scrivere su un file si possono utilizzare:

- `FileWriter`: scrive caratteri su un file.
- `BufferedWriter`: migliora l'efficienza della scrittura.
- `PrintWriter`: permette di scrivere facilmente righe con `println()`.

```

1 import java.io.PrintWriter;
2 import java.io.File;
3 import java.io.IOException;
4
5 public class ScritturaFile {
6     public static void main(String[] args) throws IOException {
7         PrintWriter writer = new PrintWriter("output.txt");
8         writer.println("Ciao mondo!");
9         writer.println("Altra riga");
10        writer.close();
11    }
12 }

```

Listing 11: Esempio di scrittura su file.

Per leggere un file si possono usare:

- `FileReader` con `BufferedReader`, per leggere riga per riga.
- `Scanner`, per leggere linee o token facilmente.

```

1 import java.io.BufferedReader;
2 import java.io.FileReader;
3 import java.io.IOException;
4
5 public class LetturaFile {
6     public static void main(String[] args) throws IOException {
7         BufferedReader reader = new BufferedReader(new FileReader("input.txt"));
8         String line;
9         while ((line = reader.readLine()) != null) {

```

```

10         System.out.println(line);
11     }
12     reader.close();
13 }
14 }

```

Listing 12: Esempio di lettura da file con BufferedReader.

```

1 import java.util.Scanner;
2 import java.io.File;
3 import java.io.IOException;
4
5 public class LetturaFile {
6     public static void main(String[] args) throws IOException {
7         Scanner scanner = new Scanner(new File("input.txt"));
8         while (scanner.hasNextLine()) {
9             String line = scanner.nextLine();
10            System.out.println(line);
11        }
12        scanner.close();
13    }
14 }

```

Listing 13: Esempio di lettura da file con Scanner.

Quando si apre un file, il metodo deve dichiarare `throws IOException` per segnalare che può generarsi un errore. È importante chiudere i file manualmente con `close()` per liberare le risorse.

Java fornisce anche utility nel package `java.nio.file` per leggere tutte le righe di un file in un colpo solo.

```

1 import java.nio.file.Files;
2 import java.nio.file.Paths;
3 import java.io.IOException;
4 import java.util.List;
5
6 public class LetturaFile {
7     public static void main(String[] args) throws IOException {
8         List<String> righe = Files.readAllLines(Paths.get("input.txt"));
9         for (String riga : righe) {
10            System.out.println(riga);
11        }
12    }
13 }

```

Listing 14: Lettura di tutte le righe con `Files.readAllLines`.